

STUDENT PERFORMANCE PREDICTION

Project handbook for learning, viva preparation, technical discussion, and self-introduction

Project in one sentence

A machine-learning regression project that learns from a student-performance CSV, compares three regression models, selects the model with the best test R-squared score, and exposes it through a Gradio prediction interface.

What this handbook is based on

This document is grounded in the submitted notebook **STUDENT_PERFORMANCE.ipynb**. It distinguishes what was actually implemented from reasonable future improvements, so you can explain the project accurately.

Notebook scope: data is uploaded in Google Colab, processed in memory, evaluated on one train/test split, then served through a temporary public Gradio link. The uploaded CSV itself and the trained model file are not stored in the project folder.

Quick facts

Problem type	Supervised machine learning - regression
Prediction target	Grade
Selection metric	Test R-squared (R ²)
Reported best model	Random Forest Regressor
Reported test performance	R ² = 0.1269; MSE = 4.7610
User interface	Gradio web interface
Execution environment	Google Colab

1. Project story and problem framing

The project answers a practical question: **given the available information about a student, what grade is the model likely to predict?** Rather than assigning a fixed rule, the system learns patterns from historical rows in a CSV file.

Each row represents one student record. The column named **Grade** is the output to predict. Every other column becomes an input feature. The notebook does not hard-code the full list of input columns; it discovers them from the uploaded dataset. One feature explicitly handled in the UI is **Weekly_Study_Hours**.

Why regression?

The project predicts a numeric grade, so it uses regression algorithms. If the goal were only categories such as pass/fail or low/medium/high performance, it would instead be a classification project.

End-to-end workflow

1. Upload	A CSV is uploaded in Colab and read with pandas.	Brings project data into the notebook.
2. Clean	Rows containing missing values are removed with dropna().	Ensures training code receives complete rows.
3. Encode	Every text/object column is converted to integer labels with LabelEncoder.	Regression models require numeric input.
4. Define data	X = all columns except Grade; y = Grade.	Separates predictors from the answer.
5. Scale	StandardScaler is fitted to X and transforms X.	Puts numeric features on a comparable scale.
6. Split	80% train / 20% test, random_state=42.	Holds out data for evaluation.
7. Train	Three models fit the same training partition.	Enables fair comparison.
8. Evaluate	Predictions on test data are scored with R2; MSE is reported for the selected model.	Measures generalization on held-out rows.
9. Serve	The selected model is called from a Gradio interface.	Lets a user enter a new student profile.

2. Data preparation and feature handling

The data-processing pipeline is simple and dynamic: it adapts to the columns in the uploaded CSV, provided that a **Grade** column exists.

Implemented transformations

Missing values	<code>df.dropna()</code>	I removed incomplete records instead of imputing values.
Categorical data	One <code>LabelEncoder</code> per object-type column	I transformed text categories into numeric codes and kept the encoders for later UI input conversion.
Feature / target split	<code>X = df.drop('Grade'); y = df['Grade']</code>	Grade is never used as an input feature.
Scaling	<code>StandardScaler</code> on X	I standardized input features before the train/test split in the notebook.
Data split	<code>train_test_split(test_size=0.2, random_state=42)</code>	I reserve 20% of rows for testing and fix the random seed for repeatability.

Important technical note for a strong explanation

The current notebook fits `StandardScaler` on the full feature dataset before the train/test split. That lets information from the test distribution influence scaling, which is called data leakage. A stronger version would split first, fit the scaler only on `X_train`, and then transform `X_train` and `X_test`. This does not invalidate the prototype, but it is an important limitation to acknowledge.

Encoding details

`LabelEncoder` assigns an integer code to each category in a text column. The notebook saves each encoder in a dictionary, then uses the same encoder during prediction. This consistency is essential: the UI must use exactly the mappings learned during training.

Caution: label encoding gives categories a numeric order that may not be meaningful. Tree models can often work with it, but for linear regression, one-hot encoding is usually more appropriate for nominal categories. Unknown categories submitted later will cause `LabelEncoder.transform()` to fail unless handling is added.

3. Models, selection, and evaluation

All three models train on the same training set and predict the same held-out test set. The project selects the highest R2 score as the best model.

Linear Regression	A linear relationship between features and Grade.	A simple baseline that is fast and interpretable.
Random Forest Regressor	An ensemble of decision trees; averages their predictions.	Can capture non-linear patterns and feature interactions.
XGBRegressor	Gradient-boosted decision trees trained sequentially to correct earlier errors.	A powerful boosting-based alternative; configured with <code>eval_metric='rmse'</code> .

Observed evaluation results

Linear Regression	-0.072443	Worse than simply predicting the test-set mean.
Random Forest	0.126913	Highest R2 among the three; selected by the notebook.
XGBoost	-0.110290	Worse than the mean baseline on this split.

Final reported metrics for the selected model

R2	0.1269128216310509	The model explains about 12.7% of variance on this test split.
MSE	4.760972413793104	Average squared prediction error; lower is better, but its scale is grade-squared units.

Honest conclusion

Random Forest is the best of the implemented candidates, but the R2 is low. The model is a working baseline, not a highly accurate production predictor. The likely next steps are better data quality, more informative features, cross-validation, tuning, and leakage-safe preprocessing.

4. Model interpretation and application module

Feature importance

After comparison, the notebook separately trains a new RandomForestRegressor and reads its `feature_importances_`. It makes a bar chart ranking the input columns by how much they helped reduce impurity across that forest's trees. This is a useful global importance view, not a statement that one feature causes a grade.

One implementation nuance: the feature-importance forest is newly trained with default randomness, rather than reusing `best_model`. It will usually be similar, but its importance values are not guaranteed to match the selected Random Forest exactly. For a polished version, set `random_state` and reuse or persist the selected model.

Gradio prediction module

Dynamic fields	The interface creates an input component for every feature column except Grade.
Categorical inputs	Dropdown choices come from the saved LabelEncoder classes.
Study-hours input	Weekly_Study_Hours is displayed as a numeric field.
Other inputs	All remaining features are displayed as text boxes in the notebook.
Prediction path	Inputs -> one-row DataFrame -> categorical encoding -> StandardScaler transform -> <code>best_model.predict()</code> -> numeric predicted grade.
Deployment	<code>iface.launch(share=True)</code> creates a temporary public Gradio share URL from Colab.

What to mention if asked about deployment

This is a prototype deployment. The notebook launches Gradio directly from Colab and uses a temporary share URL. It does not save the model, scaler, encoders, or feature schema to disk. For persistent deployment, save those assets together and host the app on a stable service such as Hugging Face Spaces or a cloud platform.

5. Tools and technologies

Python	Programming language for data processing, modelling, evaluation, visualization, and application logic.
Google Colab	Notebook environment; receives the uploaded CSV through google.colab.files.
pandas	Loads the CSV and manages tabular data / prediction rows.
NumPy	Imported for numerical work (not directly used in the shown notebook logic).
scikit-learn	train_test_split, LabelEncoder, StandardScaler, LinearRegression, RandomForestRegressor, R2, and MSE.
XGBoost	XGBRegressor candidate model.
Matplotlib	Creates figure containers for plots.
Seaborn	Builds the model-comparison and feature-importance bar charts.
Gradio	Builds the browser-based grade prediction interface.

Architecture in words

CSV data feeds the preparation layer. The prepared data is split into training and test partitions. Three model candidates are trained and scored. The highest-R2 model becomes the prediction model. The user interface applies the saved encoding and scaling steps before sending new inputs to that model.

Reproducibility status

The train/test split uses random_state=42, but RandomForestRegressor and XGBRegressor are instantiated without a random_state. Exact model results may change between runs. Set random_state on both models and record package versions for reproducible experiments.

6. How to explain the project

30-second self-introduction version

My project is a Student Performance Prediction system built with Python and machine learning. I use a CSV dataset where Grade is the target variable. I clean missing records, encode categorical values, standardize features, and compare Linear Regression, Random Forest, and XGBoost. On my held-out test split, Random Forest performed best with an R2 of about 0.127 and an MSE of about 4.76. I also built a Gradio interface so a user can enter student details and get a predicted grade.

One-minute walkthrough

The goal is to estimate a student's numeric grade from the other columns in the dataset. First, I upload the CSV in Google Colab and remove rows with missing data. Since machine-learning models need numbers, I label-encode text columns and retain those encoders for the application. I separate Grade as y and use every other column as X. Then I standardize the features, use an 80/20 split, and train three regressors. I compare them using R2 on the test set and choose the model with the best R2. Random Forest won on this dataset split. I show a feature-importance chart for Random Forest, and Gradio converts user inputs using the same preprocessing before returning the predicted grade. The result is a complete prototype, though I would improve it with leakage-safe preprocessing, cross-validation, tuning, and model persistence.

Common viva / interview questions

Why did you use regression?	Grade is treated as a numeric continuous target, so regression predicts its value directly.
Why compare models?	Different algorithms capture patterns differently. Comparison on the same test split gives an evidence-based choice.
Why was Random Forest selected?	It achieved the highest test R2 among Linear Regression, Random Forest, and XGBoost in this run.
What does negative R2 mean?	It means the model performed worse than predicting the average target value for every test row.
What does MSE measure?	It averages squared differences between actual and predicted grades; smaller values indicate lower error.
What is feature importance?	It is a Random Forest estimate of which inputs most helped split the data; it indicates association, not causation.
What is the main limitation?	Low R2 and a preprocessing leakage risk from scaling before splitting; this is a baseline prototype.

7. Improvement roadmap and accurate boundaries

Recommended next iteration

High	Split before fitting preprocessing; use a Pipeline.	Prevents data leakage and makes inference consistent.
High	Add random_state to Random Forest and XGBoost; save package versions.	Makes results repeatable.
High	Use cross-validation and report mean / spread of R2, MAE, RMSE.	Avoids relying on one lucky or unlucky split.
High	Persist model, scaler, encoders, and ordered feature list with joblib.	Creates a reliable reusable application artifact.
Medium	Use ColumnTransformer and OneHotEncoder for nominal categories.	Improves handling of text features, especially for linear models.
Medium	Tune hyperparameters using GridSearchCV or RandomizedSearchCV.	May improve generalization.
Medium	Add input validation and unknown-category handling in Gradio.	Prevents user-input failures.
Medium	Add MAE and RMSE alongside MSE.	Provides errors in more interpretable grade units.

Do not overclaim

The submitted notebook does **not** show the original CSV contents, dataset size, exact feature list, data source, or permanently saved model artifacts. It also does not show hyperparameter tuning, cross-validation, fairness testing, or production hosting. Describe those only as future work unless you add and verify them.

Final takeaway

This project demonstrates the full beginner-to-intermediate ML workflow: dataset ingestion, cleaning, categorical handling, scaling, regression-model comparison, metric-based selection, feature-importance visualization, and a usable prediction interface. Its central lesson is not that the current model is highly accurate; it is that you can build, evaluate, and honestly improve an end-to-end prediction system.

Prepared from the project notebook on 04 Sep 2026